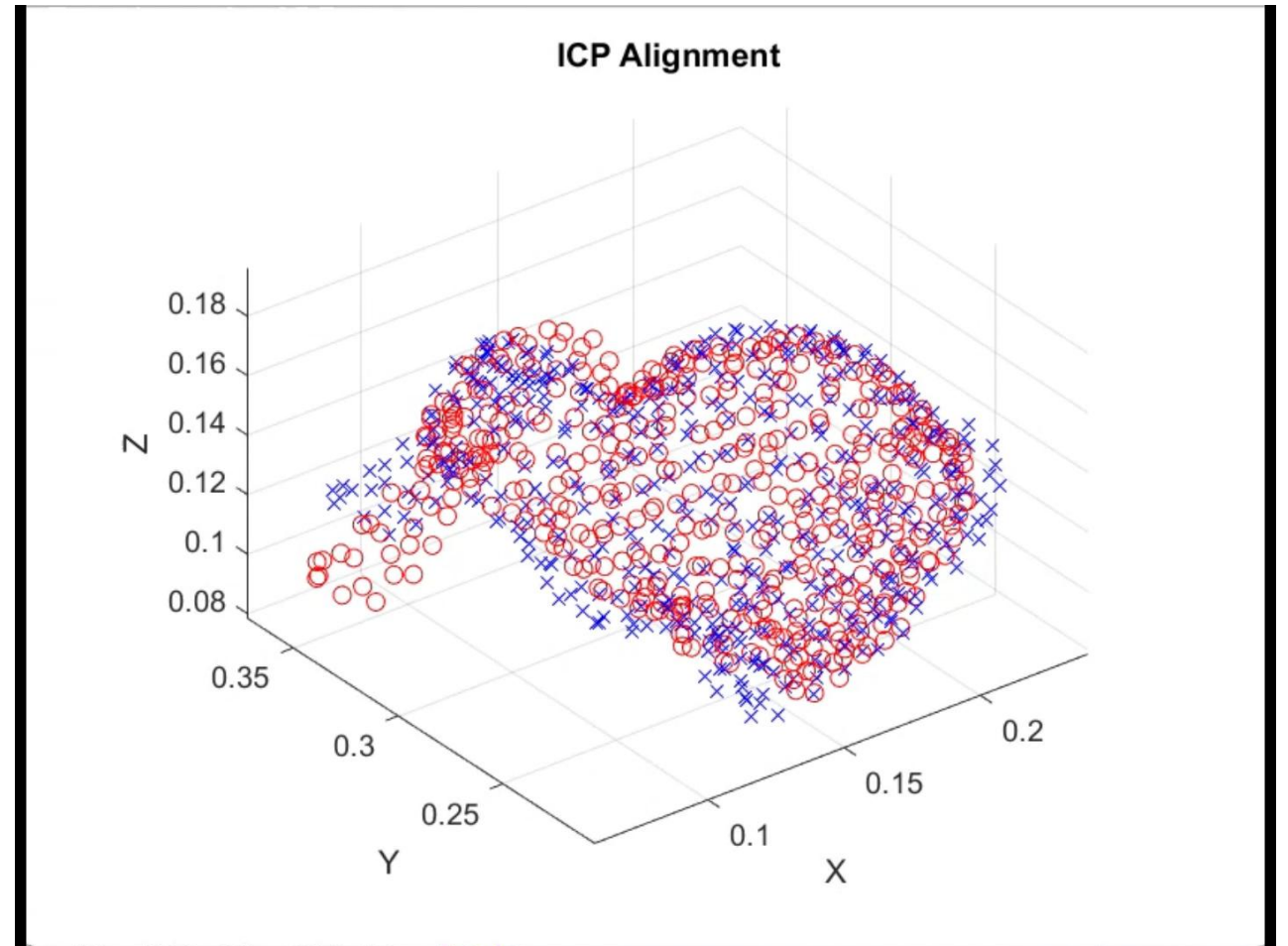


ICP

VLP-16 Sensor Data

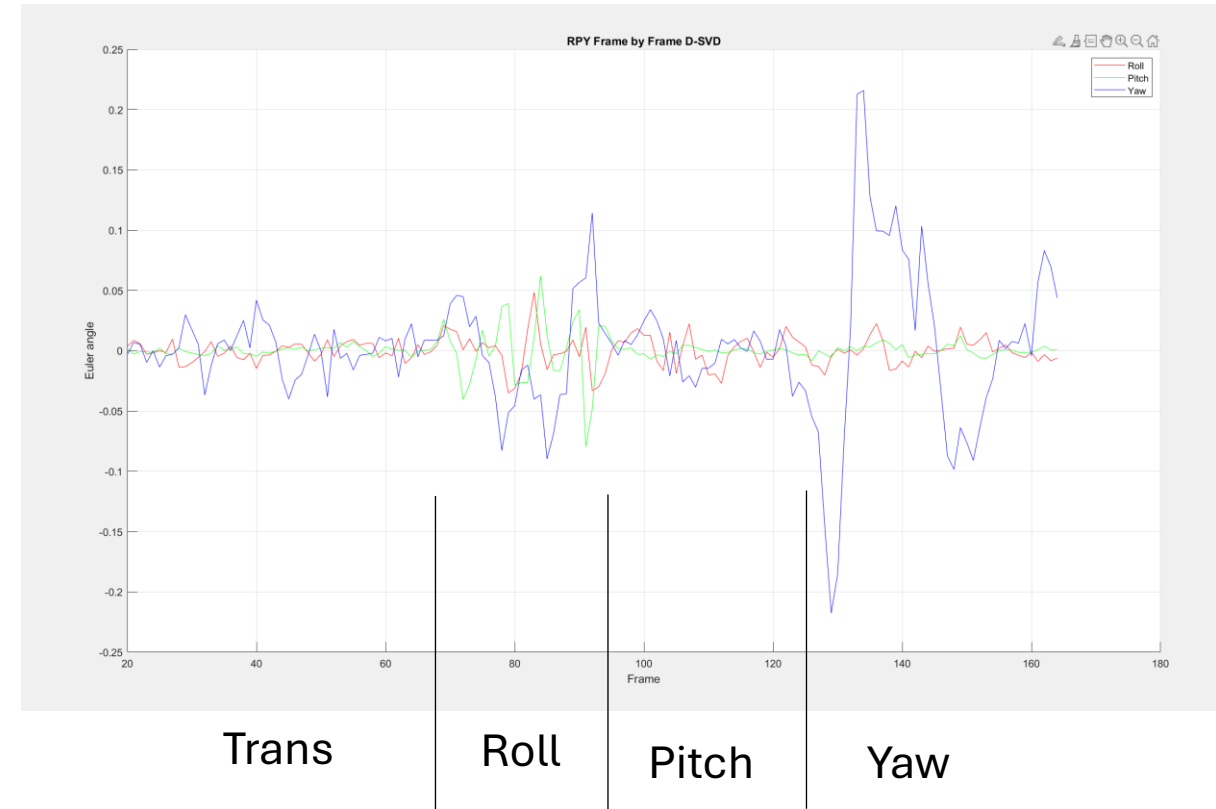
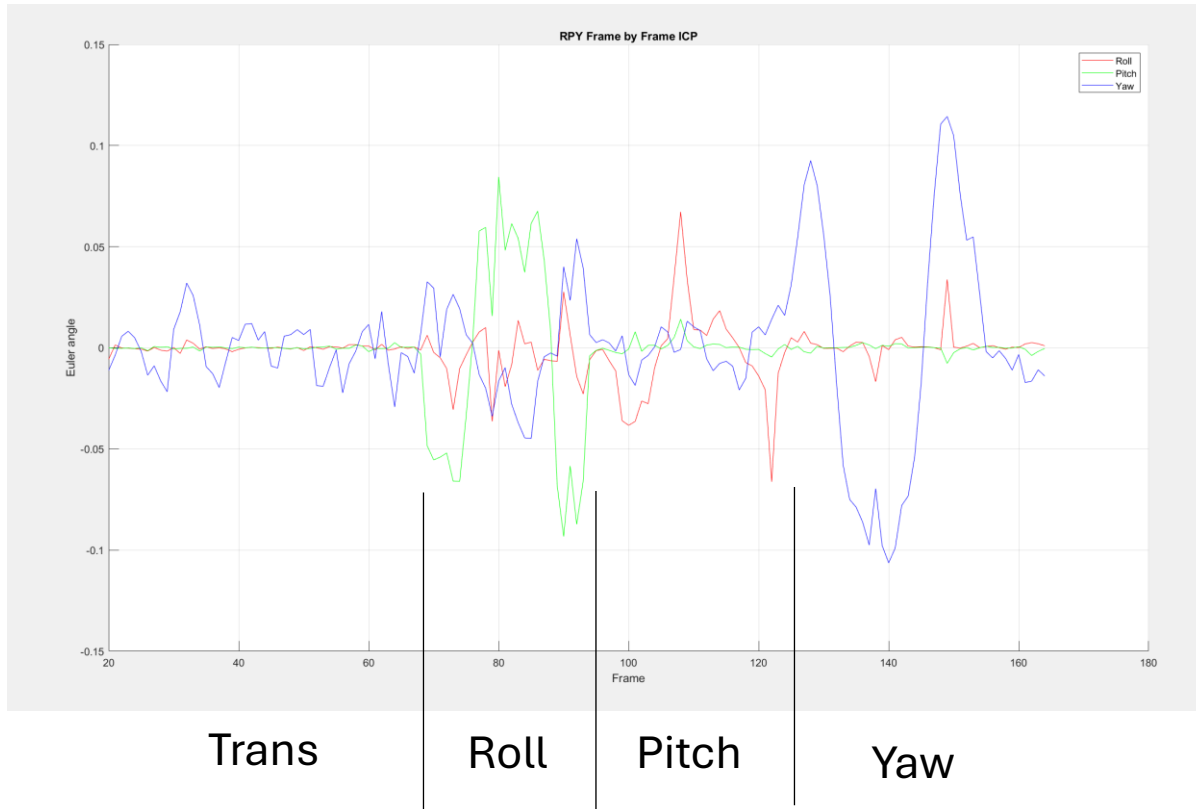
ICP Algorithm

- Based on Peter Corke's model
 - Except rotation estimated before translation
- Repeated iterations until certain minimum "error" reached or reaches preset max number of iterations
- Tested with Stanford bunny
 - Successful
- Testing with VLP 16
 - Hit or miss
 - Working on filtering, noise reduction



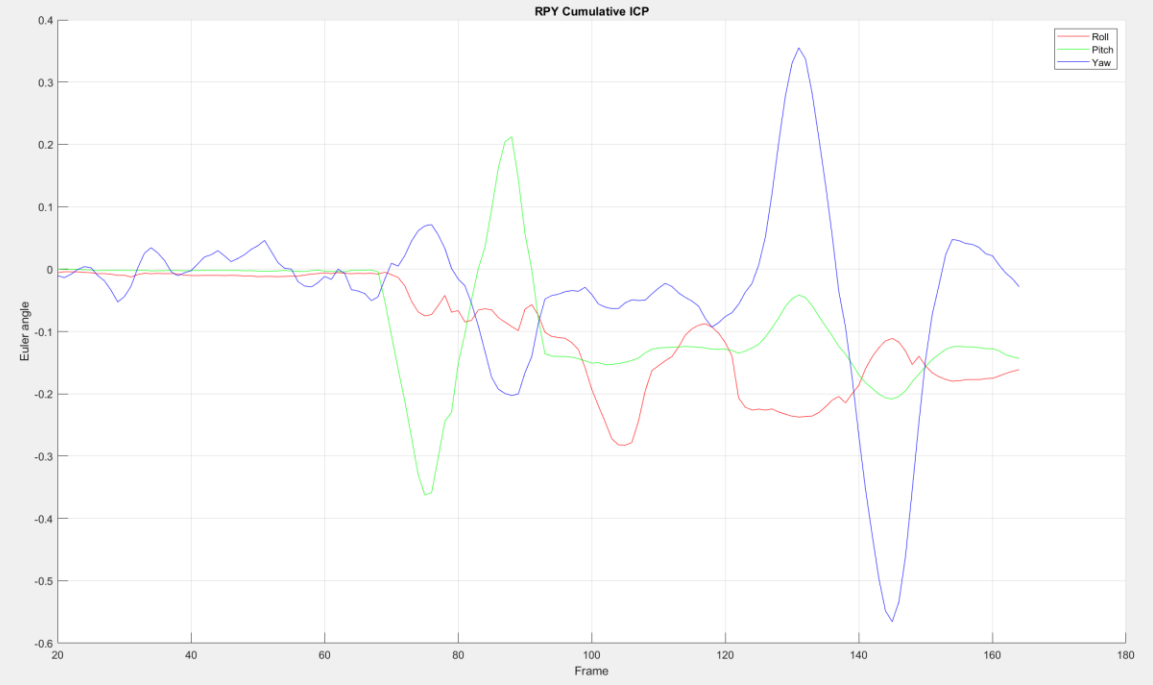
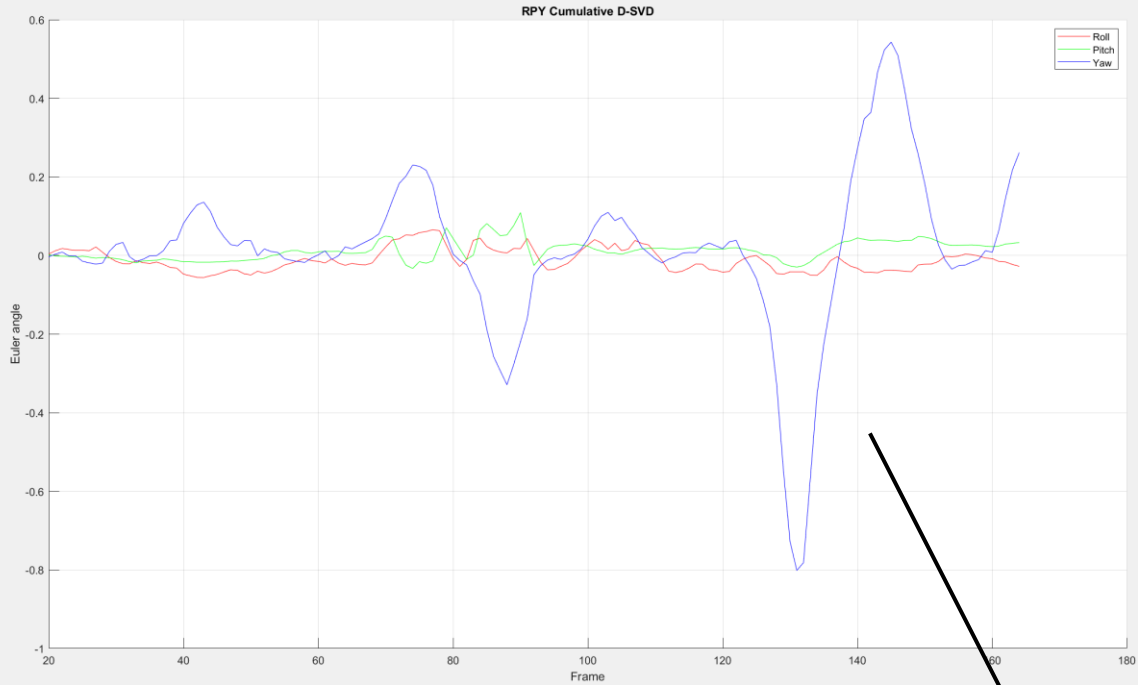
(Video)

ICP and D-SVD Compared Frame by Frame



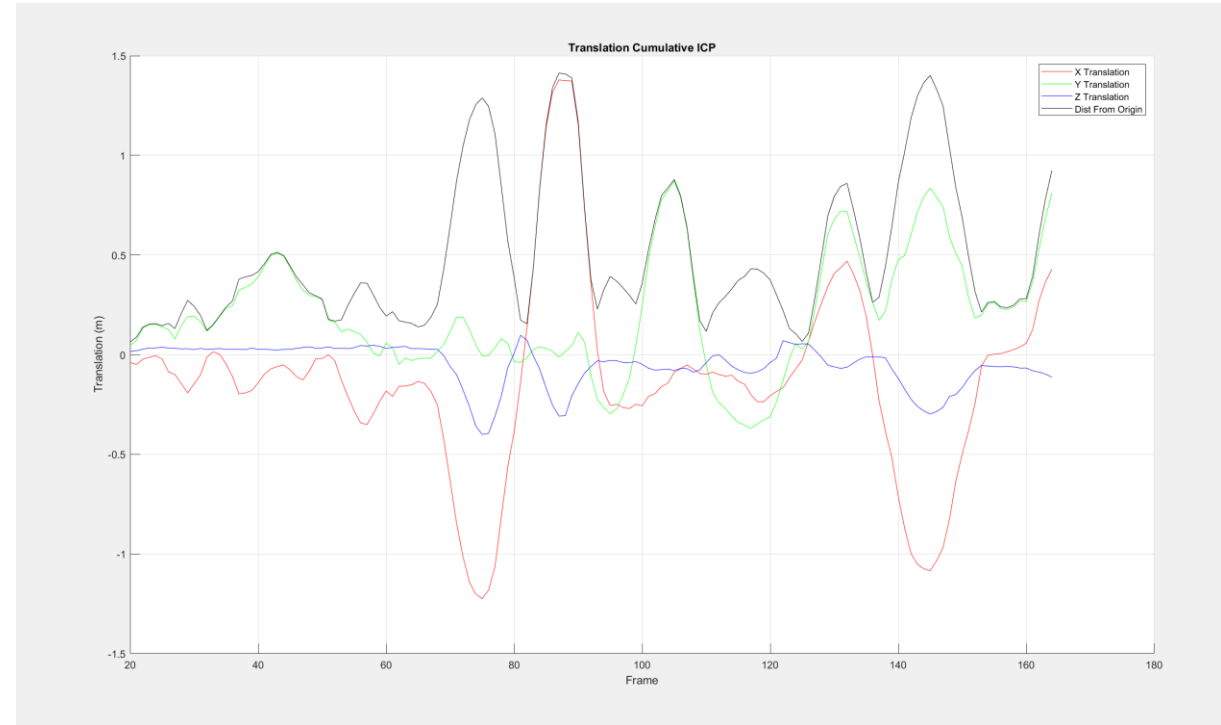
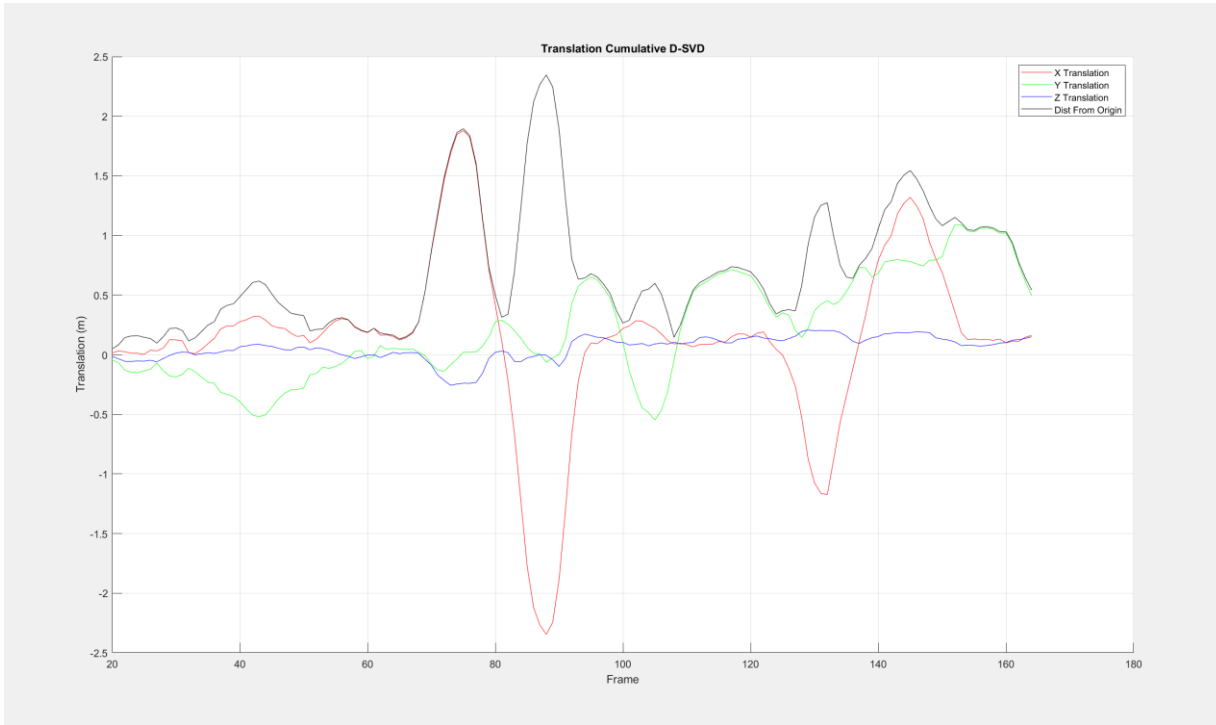
- For roll, pitch, and yaw, the sensor was moved *both* directions (clockwise and counterclockwise)
- ICP: max of 50 iterations with an unattainable error bound set

ICP and D-SVD Compared Cumulatively



Correct yaw

Translation



Exploring ICP (with the Stanford bunny)

To test the robustness of his algorithm, Peter Corke used the lines:

```
>> D(:, randperm(numcols(D), 40)) = [];
```

to randomly remove 40 unique points from the data

```
>> D = [D 0.1*rand(3,20)+0.1];
```

to add back 20 fake points

```
>> D = D + 0.01*randn(size(D));
```

to add Gaussian noise with standard deviation of 0.01

Exploring ICP Further

```
>> D = D + n*randn(size(D));
```

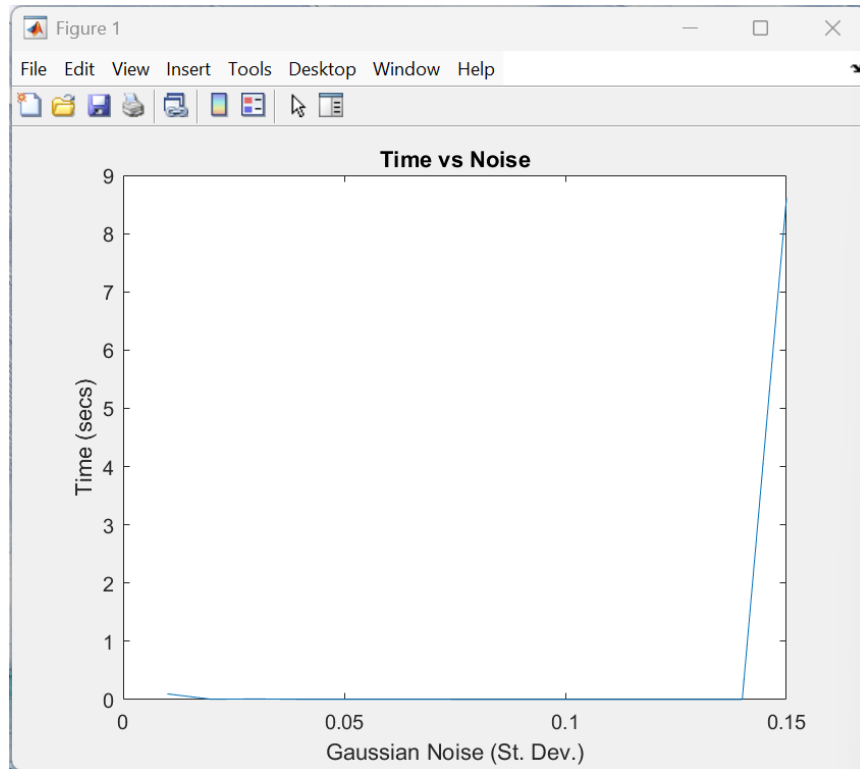
n: 0.01 to 0.15

- Essentially a controllable amount of noise
- Driving Question: What impact is there on the error in the program's pose estimation and program runtime?
- Presumably, more noise greatly increases the number of iterations and time needed to approach an error
- More noise would mean more error in estimated translation

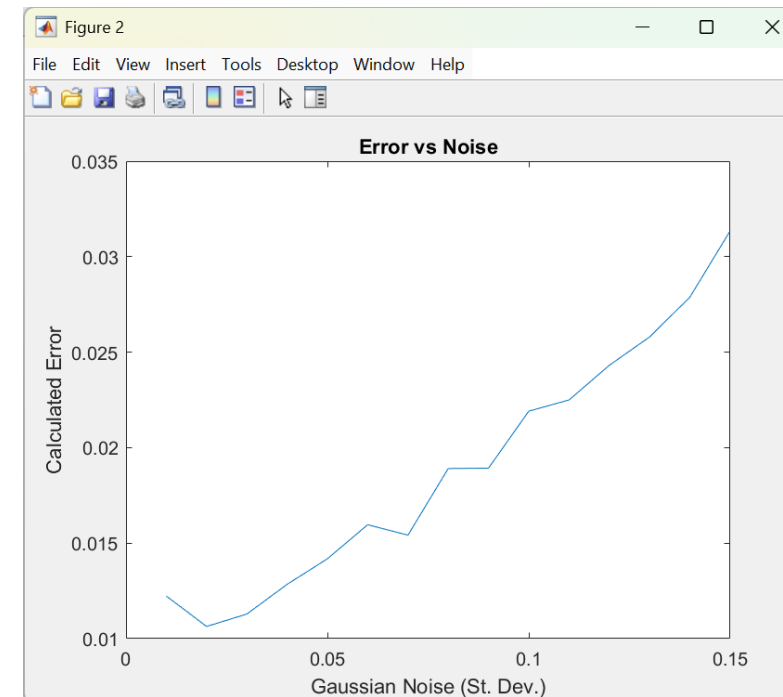
Testing and Results

- Modifying Corke's code, manipulate the bunny point cloud with Gaussian noise of standard deviation = n
- n goes from 0.01 to 0.15

Initially, most of my time graphs looked like this:



because as noise increases, error increases $\sim$ linearly, making it hard to set an error limit for the algorithm to approach as it will either instantly reach it or never reach it, with little in between



Conclusion

- Those sharp increases in time graphs imply that rather than time increasing as error increases, it seems that an ICP algorithm will be able to run efficiently and almost immediately produce a relatively accurate pose estimate or will *never* obtain a good estimate.
- Application?
 - Use ICP for smaller pose estimations